Smart Traffic Signal Optimization

Machine learning and computer vision | Two-page revision guide

01 Project objective

Reduce waiting at a simulated four-way intersection by adjusting signal phases to current queues and predicted arrivals. Compare **fixed-time**, **queue-only adaptive**, and **ML-assisted adaptive** controllers on identical traffic.

02 The ML model: Random Forest Regression

This is **supervised regression**: the model learns from examples with numerical targets. It predicts demand; a separate controller chooses the signal phase.

Input features	Prediction target
Vehicle arrivals in the last three completed 10-second intervals, newest first.	Vehicle arrivals in the next 10 seconds.

Example: input [6, 6, 6] produces approximately **5.61 arrivals**. Fractional predictions represent expected counts.

03 Dataset and training algorithm

Traffic is generated using Poisson arrivals with changing rates. Training uses **300 independent episodes / 9,900 examples**; testing uses **100 separate episodes / 3,300 examples**. Each episode has 36 intervals. No episode appears in both sets.

Training steps: (1) generate episodes; (2) turn three past counts into features and the following count into the target; (3) fit the forest; (4) predict on held-out episodes; (5) measure errors against simple baselines.

Each tree learns threshold splits that reduce squared prediction error from a bootstrap sample. A leaf predicts the average training target reaching it. The forest averages the outputs of its **60 trees**. No neural-network epochs or backpropagation are used.

Setting	Value / purpose
Trees / maximum depth	60 / 8; limits model complexity
Minimum leaf size	15 examples; reduces overly specific predictions
Random state / workers	9 / 1; reproducible training
Implementation	Python + scikit-learn; trained at server startup

Control, vision & results

04 Signal-control algorithm

For each approach, the controller calculates:

Priority = queue + 0.7 x predicted arrivals + 0.08 x time since green

A larger score generally receives higher priority. The weights are manually chosen, not learned. Predictions are refreshed every 10 seconds using only past arrival counts.

Timing rules: green lasts 10-45 seconds; phases have a 3-second no-discharge transition. A normal early switch requires a competing score more than 2 points higher. Waiting approaches without green for 90 seconds get overdue priority at an eligible decision. This is not a strict limit on individual vehicle waiting.

05 Computer vision demonstration

Generated image → color segmentation → connected components → bounding boxes → approach counts.

The detector groups neighboring blue vehicle pixels and removes components smaller than 50 pixels. This is **classical image processing, not a trained object detector**. Camera visibility is capped at nine vehicles per approach. CV is demonstrated separately; the controller uses complete simulator counts.

06 Measured results

Forecast quality: held-out mean absolute error (MAE) is **1.704 vehicles per 10 seconds**; RMSE is 2.203. Baseline MAEs: three-window average 1.733; latest-count persistence 2.061; constant training mean 2.198. The gain over the moving average is modest.

Control evaluation: three scenarios x ten seeds x three policies = 90 policy runs. Each lasts 600 seconds. Mean accumulated delay per arriving vehicle:

Scenario	Fixed-time	Queue-only	ML-assisted
Balanced	38.78 s	21.17 s	21.11 s
North/South rush	99.39 s	51.29 s	49.10 s
Eastbound surge	93.34 s	33.39 s	32.17 s

Delay = accumulated queue-seconds / arrivals. It includes waiting so far for unfinished vehicles, but excludes their future waiting after the run. Most improvement comes from adaptation; ML adds a smaller average benefit and does not win every individual run.

07 Scope and your viva explanation

Synthetic data only; one vehicle/second discharge; no turns, pedestrians, real-video validation, or road deployment. Nine automated tests pass, including past-only features and proof that forecasts change actual signal decisions.

Say: "I trained a Random Forest to predict short-term vehicle arrivals. A heuristic controller combines forecasts, queues and waiting age to choose signals. I compare policies on identical synthetic traffic and separately demonstrate image-based vehicle counting."

Evidence: src/model.py, src/simulation.py, src/vision.py; artifacts/model_verification.json and evaluation_summary.json. Reproduce with: python3 verify_model.py